NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

TinyML - Introduction to ANN(FCNN) and CNN Operations

Assoc Prof Nicholas Vun
College of Computing
and Data Science

Computer Vision Sensing

The most common Machine Learning applications are vision related

- although NLP/Generative AI has now attracted many attentions due to ….

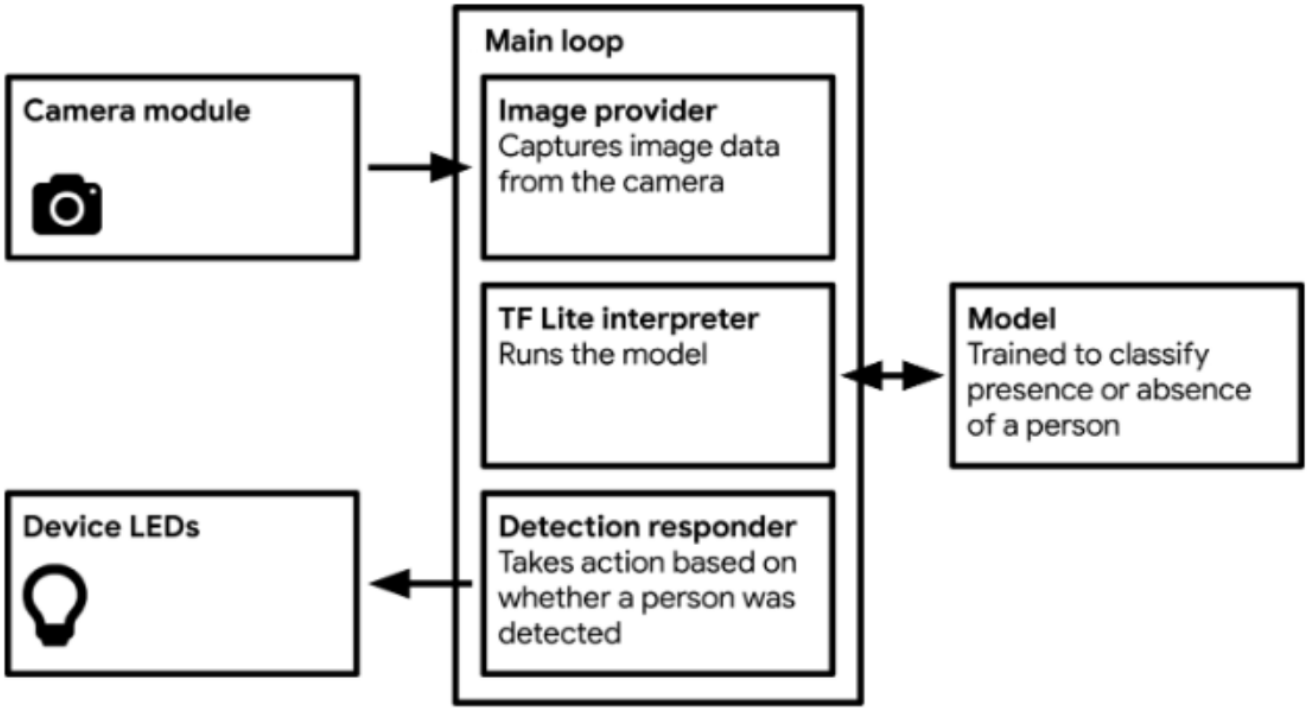
Computer Vision can be applied to a wide range of industries:

- Transportation: Autonomous Vehicles, Traffic flow analysis, Pedestrian detection
- Retail: Self-checkout, Stock replenishment, Footfall traffic, theft detection
- Healthcare: X-Ray analysis, CT scans and MRI, Movement analysis
- Manufacturing: Defect inspection, Reading text and barcodes, Product assembly, Predictive maintenance
- Agriculture: Crop and yield monitoring, Insect detection, Plant disease detection

The most common ML model for Computer Vision based applications

- Convolutional Neural Network (CNN)

An implementation of TinyML Person Detection



(Source: TinyML Pete Warden & Daniel Situnayake)

Image Classification Implementation

Image model expects its input to be an array of pixel values

- no real need to do much preprocessing before feeding data into the model (may be just rescaling of size)

Camera takes a snapshot

- feed data into the model
- determines which output class is detected
- displays the result in some simple manner.

Example:

- 96 × 96 pixel grayscale images
- image = 3D tensor with shape (96, 96, 1)
- each pixel contains an 8-bit value that represents intensity of the pixel.

Training - Visual Wake Words Dataset

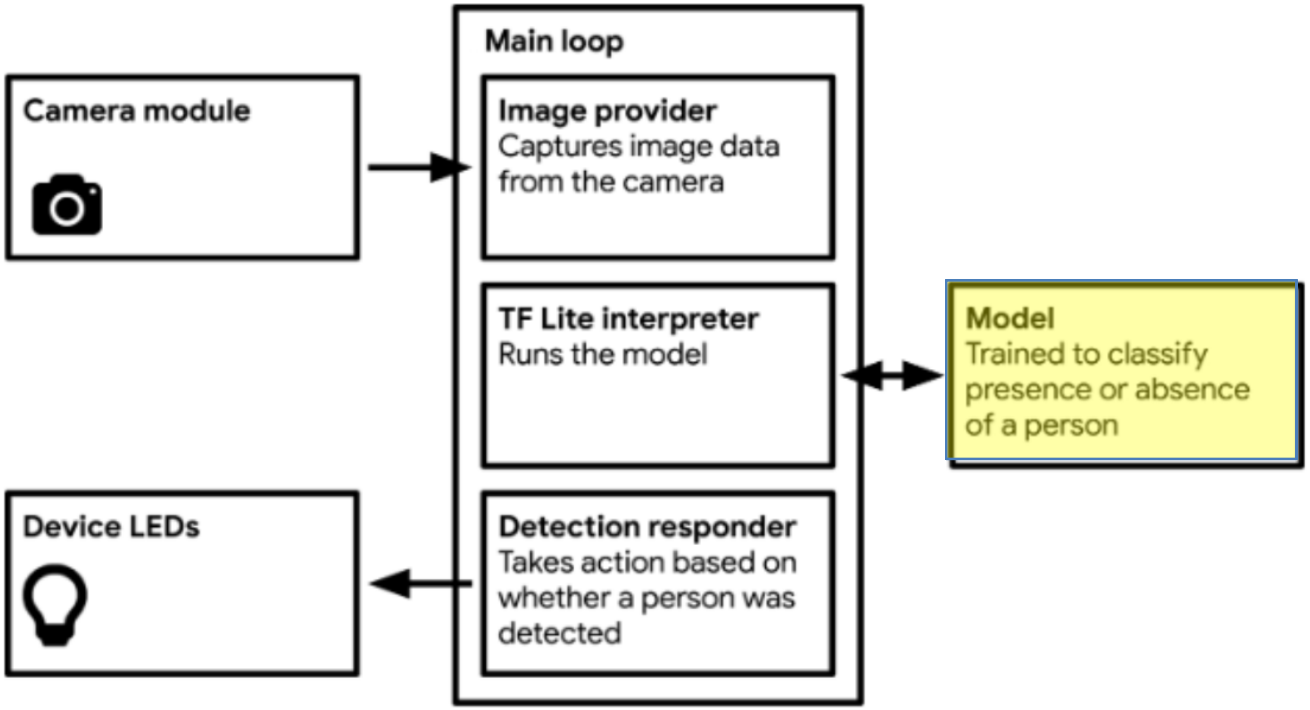
A common use-case for microcontroller vision

- to classify whether a person is present in the image or not

(a) ‘Person’

(b) ‘Not-person’

An implementation of TinyML Person Detection



(Source: TinyML Pete Warden & Daniel Situnayake)

Model Architectures Selection

Model architecture design is critical to the performance of ML

- particularly important for TinyML due to memory constraint of microcontrollers
- still an active research field

Time consuming process to find an optimum(?) design

- number of layers and neurons
- type of activation functions

Start with the simplest to avoid over design

- iterate until the performance meets specification

Best starting point

- use some existing architectures that have been proven

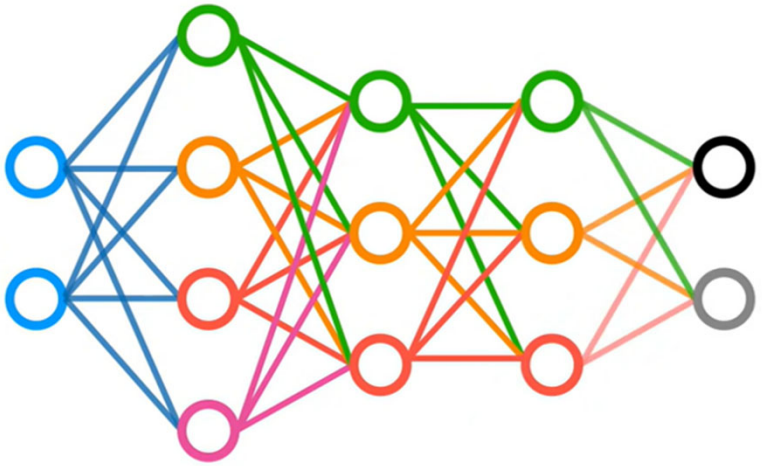
Example: for computer vision applications

- Convolutional Neural Network (CNN)

Artificial Neural Network (ANN)

Artificial Neural Network

- conventional and intuitive approach
- each neuron is connected to every other neurons in the next layer
  - Fully connected neural network (FCNN)
  - ‘Dense’ and ‘Sequential’ in Keras



```
#Model specification
model = Sequential()
model.add(Input(shape=(2,)))
model.add(Dense(units=4))
model.add(Dense(units=3))
model.add(Dense(units=3))
model.add(Dense(units=2,
                activation='softmax'))
```

FCNN for Image Classification

MNIST dataset for digit recognition

- **Modified National Institute of Standards and Technology** dataset
- a large database of handwritten digits
- grayscale images of handwritten single digits between 0 and 9.
- each of size 28x28.
- 60,000 training images and 10,000 testing images

(Wikipedia - Sample images from MNIST test dataset)

FCNN for Image Classification

Consider the following handwritten image

- greyscale (0 to 255)
- assume 7x7 size

0	0	0	0	0	0	0
0	87	240	210	24	0	0
0	13	0	101	195	0	0
0	35	167	99	210	0	0
0	145	230	240	201	189	140
0	0	102	67	17	13	0
0	0	0	0	0	0	0

0	0	0	0	0	0	0
0	87	240	210	24	0	0
0	13	0	101	195	0	0
0	35	167	99	210	0	0
0	145	230	240	201	189	140
0	0	102	67	17	13	0
0	0	0	0	0	0	0

(Source: Youtube – codebasics)

FCNN for Image Classification

Flatten the 7x7 2D array to 1D array (i.e. a feature vector)

- size = 1x49

0	0	0	0	0	0	0
0	87	240	210	24	0	0
0	13	0	101	195	0	0
0	35	167	99	210	0	0
0	145	230	240	201	189	140
0	0	102	67	17	13	0
0	0	0	0	0	0	0

$$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 87 \\ 240 \\ 210 \\ 24 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

(Source: Youtube – codebasics)

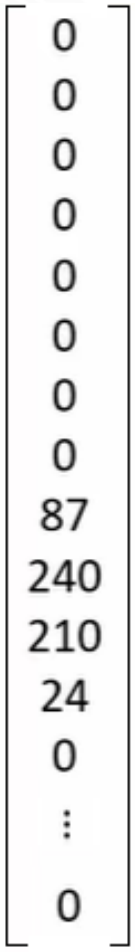
FCNN for Image Classification

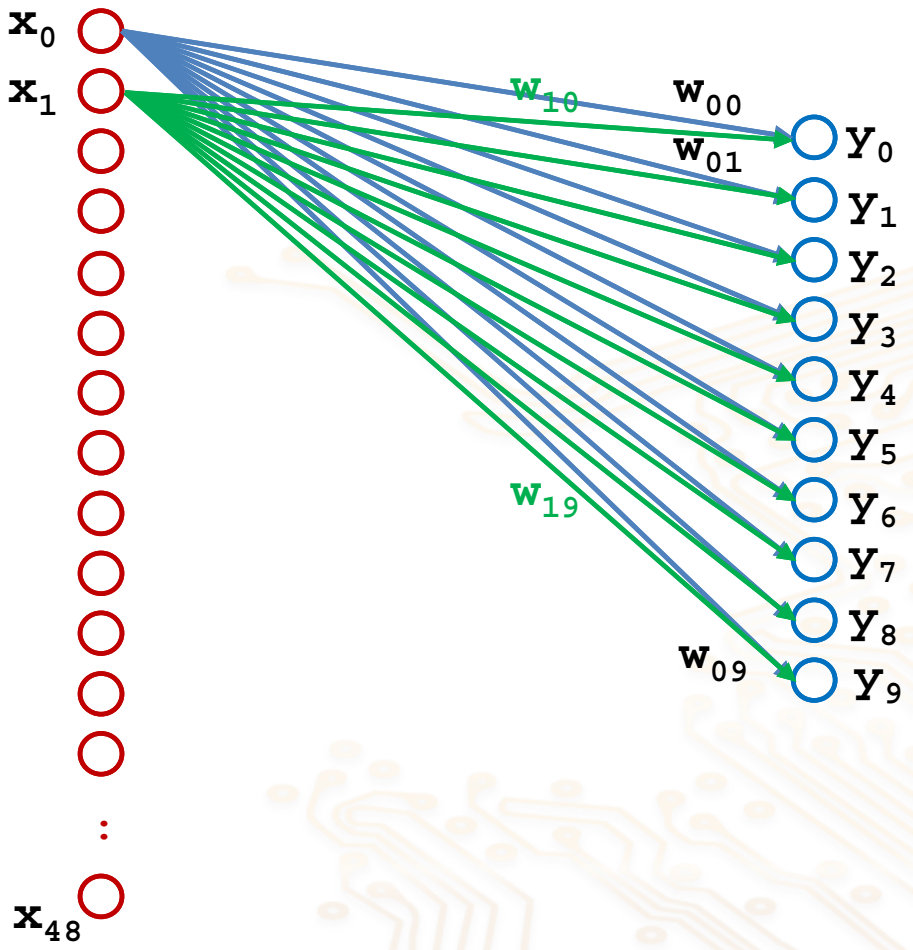
Applied to a simple FCNN

- no hidden layer
- number of input nodes = 49
- number of output nodes = 10

Fully connected

⇒ each input neuron is connected to every output neurons



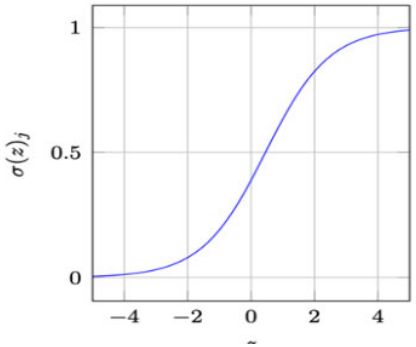


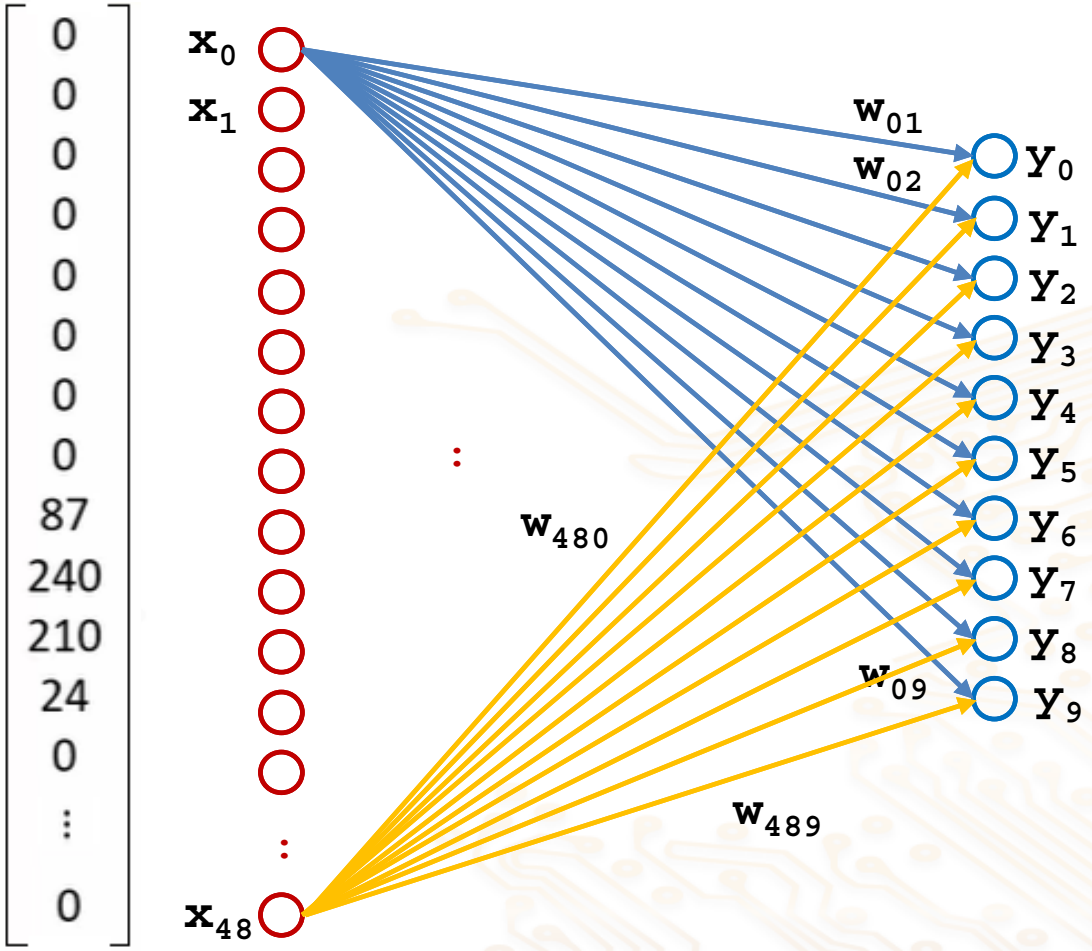
FCNN for Image Classification

- Number of connections
  = 49 x 10
  = 490
- Number of weights to adjust
  = 490

Each output node

- Softmax activation computation





Case Example: ANN for MNIST Image Recognition

We will use the Python program to code an ANN to recognize MNIST digit

- Import all necessary library

```
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt
import numpy as np
```

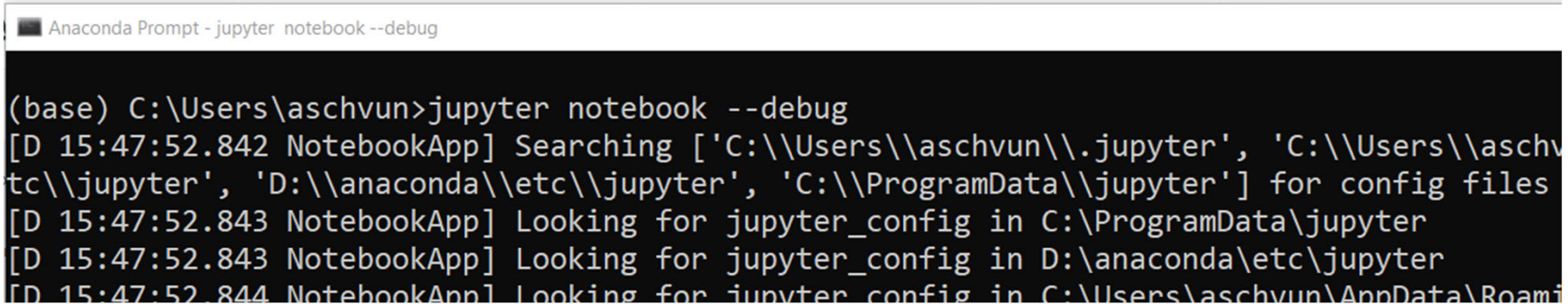
- Load the built-in MNIST dataset through Keras

```
(X_train, y_train) , (X_test, y_test) = keras.datasets.mnist.load_data()
```

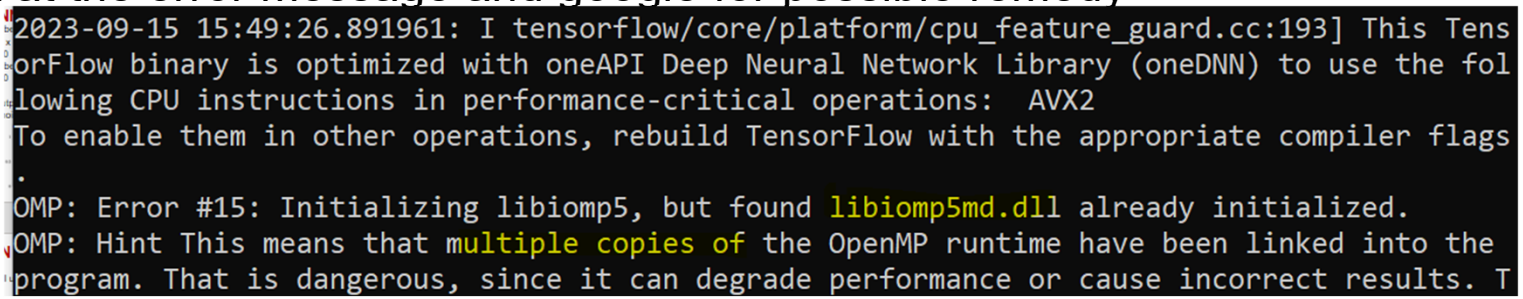
Aside: Debugging Kernel Terminate

If you encounter problem (e.g., Kernel crash) during the running of your program

- launch the Jupyter Notebook through Anaconda Prompt with debug option



- Execute the script until you see the point it crashes.
- Look at the error message and google for possible remedy



MNIST Data

- Check the size of the dataset

```
len(X_train)
```

```
len(X_test)
```

- Check the data dimension

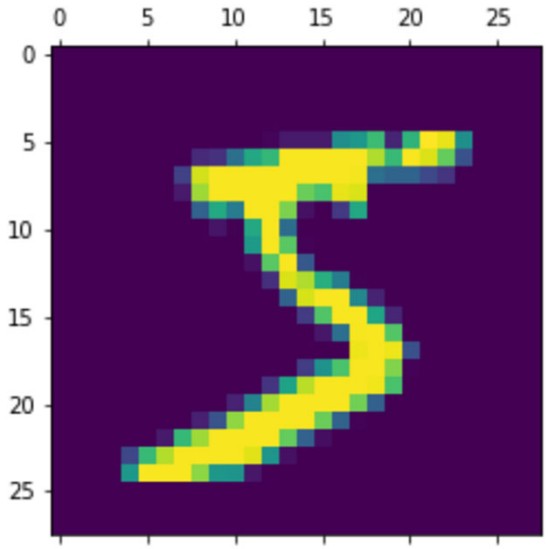
```
X_train[0].shape
```

```
X_train.shape
```

MNIST Data

- View an image

```
plt.matshow(X_train[0])
```



- Its label (for supervised training)

```
y_train[0]
```

- Check first few entries

```
y_train[:5]
```

Preprocessing the data

- Convert the data to have range with (0:1)

```
X_train[0]
```

```
X_train = X_train / 255
X_test = X_test / 255
```

```
X_train[0]
```

- Reshape the Image from 2D to 1D

```
X_train_flattened = X_train.reshape(len(X_train), 28*28)
X_test_flattened = X_test.reshape(len(X_test), 28*28)
```

```
X_train_flattened.shape
```

Setting up the Model

- A simple 1-layer model
  - 784 inputs
  - 10 outputs
  - activation function = softmax

```
model = keras.Sequential([
    keras.Input(shape=(784,)),  # Defines the input layer
    keras.layers.Dense(10, activation='softmax')
])

model.summary()
```

Layer (type)	Output Shape	Param #
dense_11 (Dense)	(None, 10)	7,850

Total params: 7,850 (30.66 KB)

Trainable params: 7,850 (30.66 KB)

Non-trainable params: 0 (0.00 B)

Training Setup

- Optimizing algorithm for training = Adam
  - Loss parameter = cross entropy
  - Measurement metric = accuracy

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

- Run the training

```
model.fit(X_train_flattened, y_train, epochs=5)
```

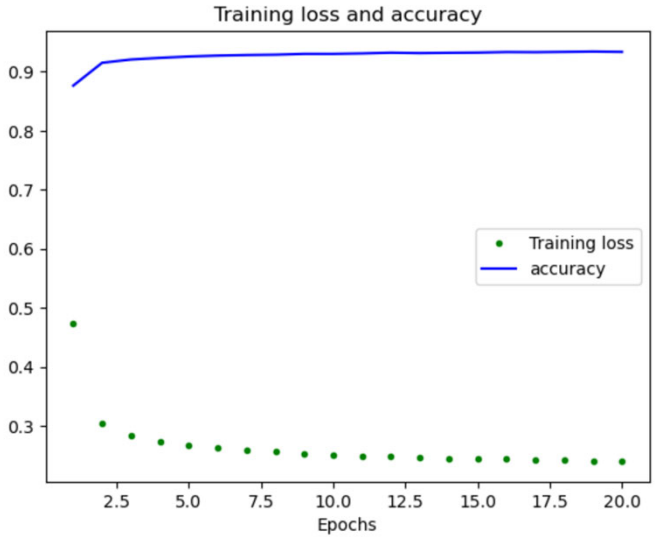
Training Performance

Plot the training's progress

```
# Draw a graph of the loss and accuracy
train_loss = history.history['loss']
accuracy = history.history['accuracy']

epochs = range(1, len(train_loss) + 1)

plt.plot(epochs, train_loss, 'g.', label='Training loss')
plt.plot(epochs, accuracy, 'b', label='accuracy')
```



Inference Performance

- Evaluate the model using the test data

```
model.evaluate(X_test_flattened, y_test)
```

- Use the model to do prediction

```
y_predicted = model.predict(X_test_flattened)
y_predicted[0] #predicted ouput based on 1st image
```

```
#check which is the biggest predicted probability
np.argmax(y_predicted[0])
```

```
#Verify against the actual input image
plt.matshow(X_test[0])
```

Inference Performance

- Process all the predicted results

```
#get all the predicted output
y_predicted_labels = [np.argmax(i) for i in y_predicted]
```

- View the first 10 predicted values

```
#see the first 10 predicted value
y_predicted_labels[:10]
```

- Compare against the actual values

```
#check against the actual first 10 value
y_test[:10]
```

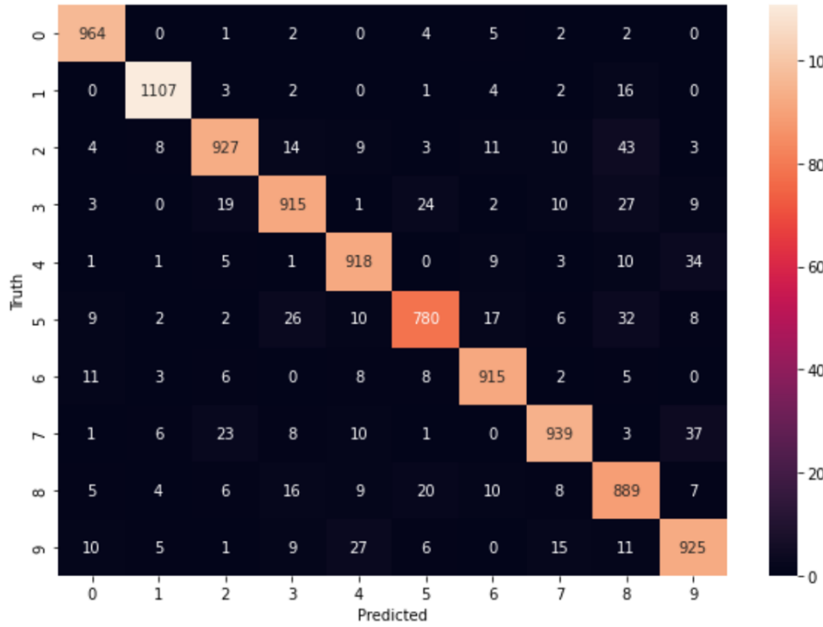
Confusion Matrix and Seaborn Heatmap

We can use the confusion matrix function to provide a summary of the number of correct/incorrect predictions for each number

```
cm = tf.math.confusion_matrix(labels=y_test,predictions=y_predicted_labels)
```

Seaborn heatmap function can then be used to visually display the accuracy of the predictions.

```
import seaborn as sns
plt.figure(figsize = (10,7))
sns.heatmap(cm,annot=True, fmt='d')
plt.xlabel('Predicted')
plt.ylabel('Truth')
```

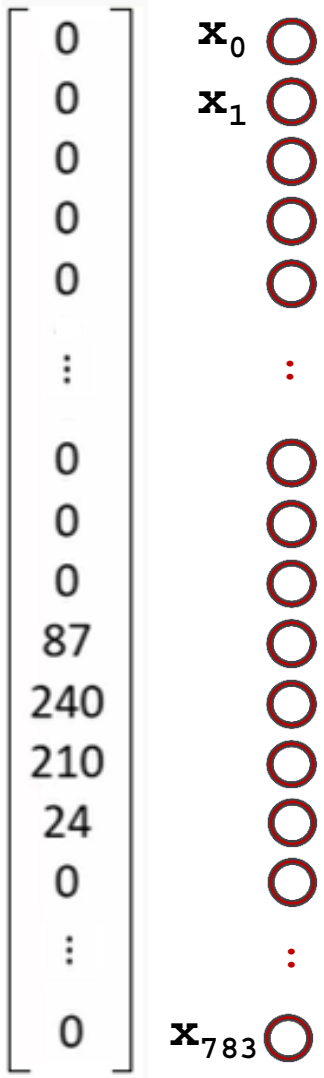


However …..

MNIST handwritten images: 28x28 size

- number of inputs = 784

0	0	0	0	0	..	0
0	87	240	210	24	...	0
0	13	0	101	195	...	0
0	35	167	99	210	...	0
0	145	230	240	201	...	140
...	...	...	...	...	...	...
0	0	0	0	0	...	0



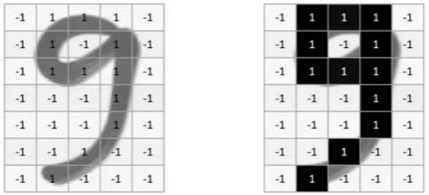
(Source: Youtube – codebasics)

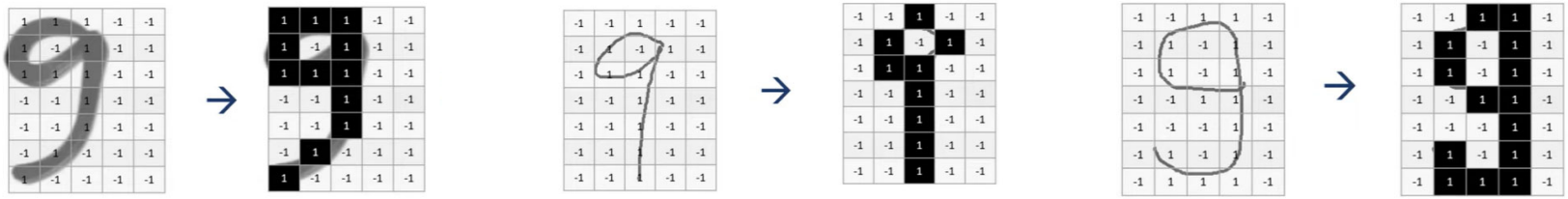
Limitation of FCNN

Not scalable – particularly important for memory constraint device

- E.g., high resolution colour image: 1920 x 1080 x 3 channels

Sensitivities to shift and variations



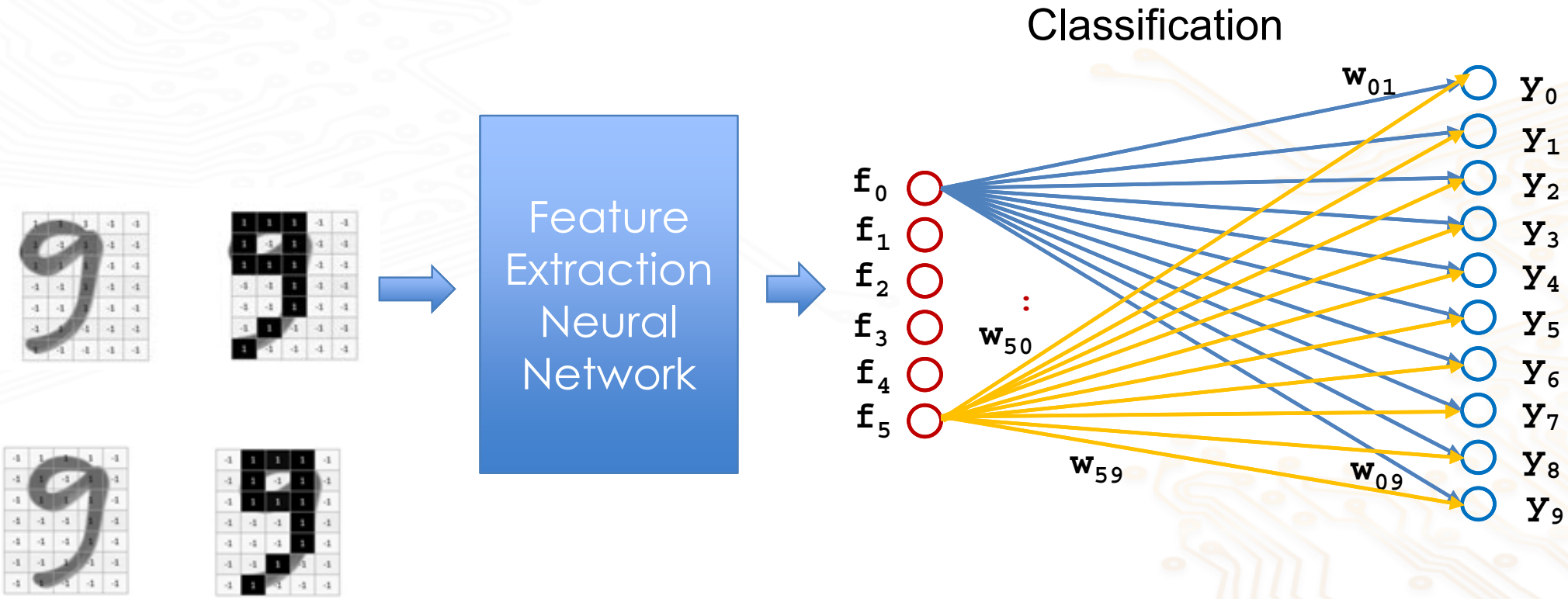


(Source: Youtube – codebasics)

‘Smarter’ Approach

Instead of performing the classification directly from the raw image

- extract distinct features from the raw image
- classified based on distinct features

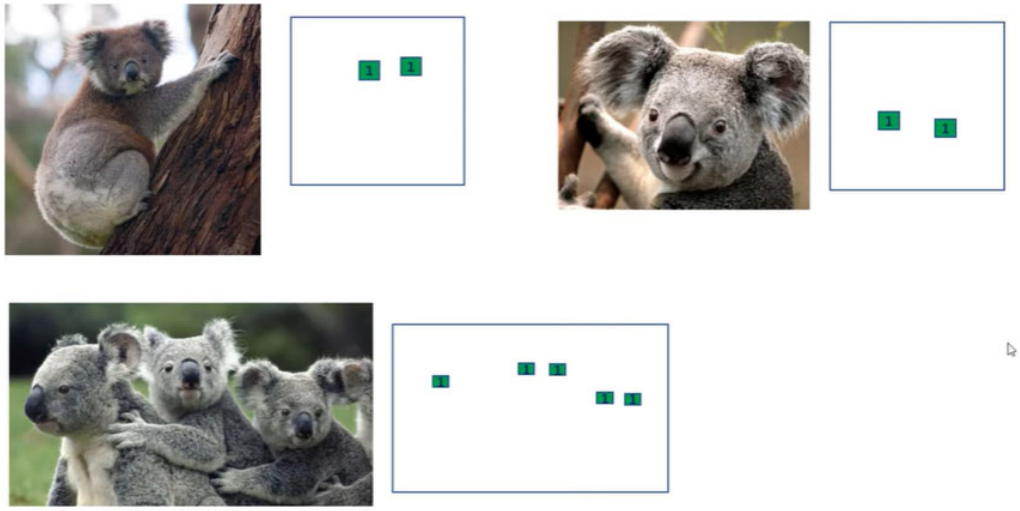


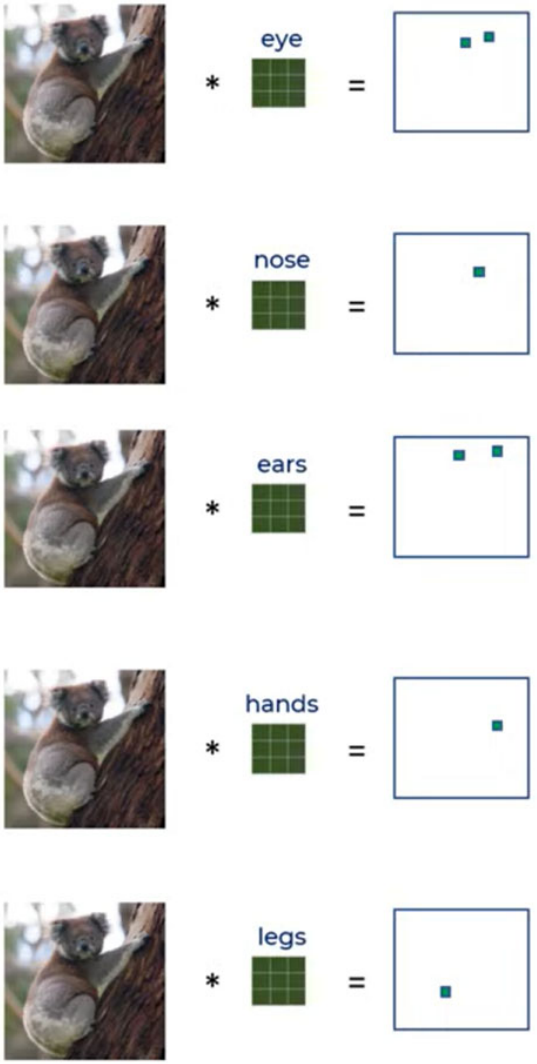
Distinct Features

Identify the distinct features

- position invariance

Example: Eyes detection





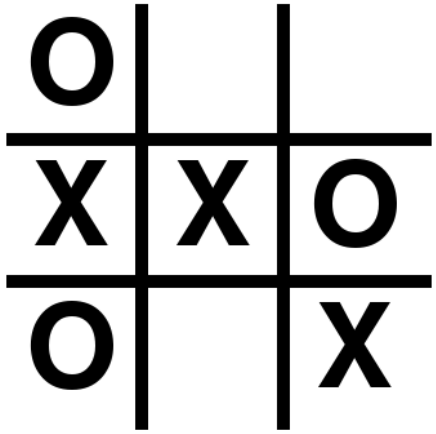
(Source: Youtube – codebasics)

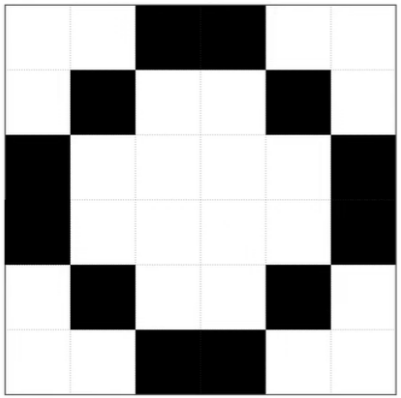
Image Classification using Features

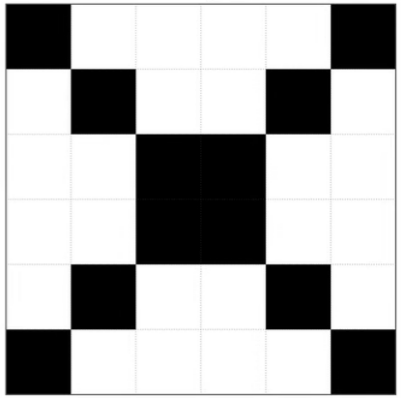
Steps:

- first extract distinct features from the images
- then classify based on distinct features detected
  - which reduce the number of inputs needed

Simple example: Classify the **X** and **O** in Tic-Tac-Toe game

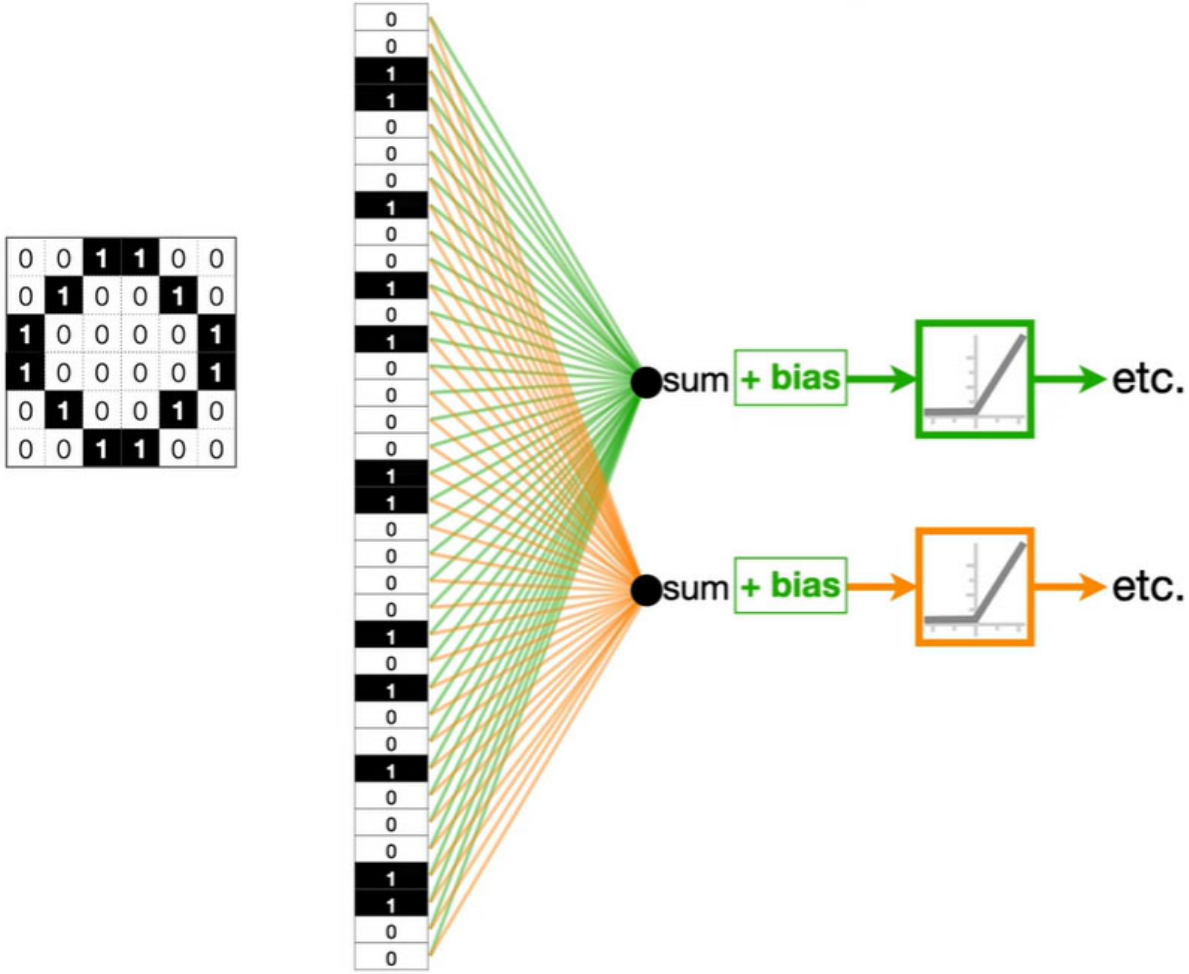






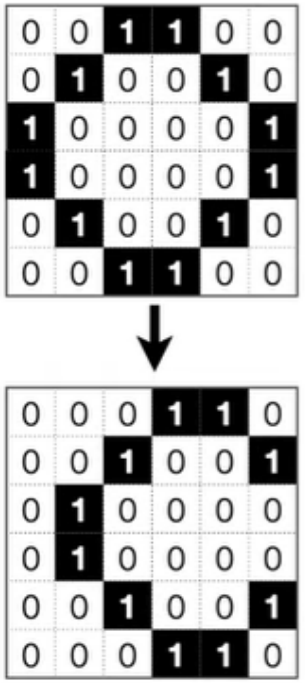
(Source: Youtube – StatQuest)

FCNN Approach

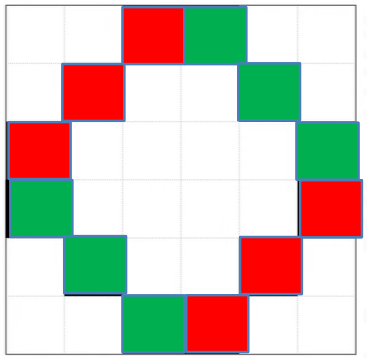


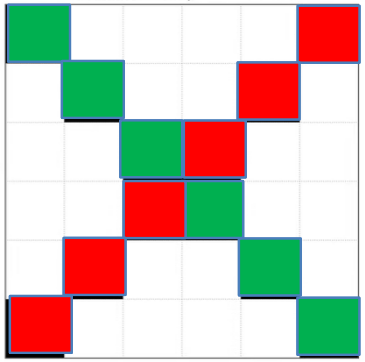
(Source: Youtube – StatQuest)

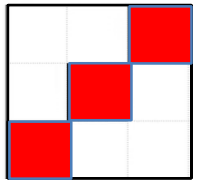
What if ?

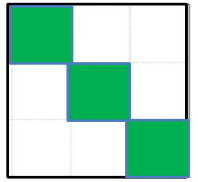


Features Detection Approach









Distinct feature?

- shapes are definitely different

Basic features

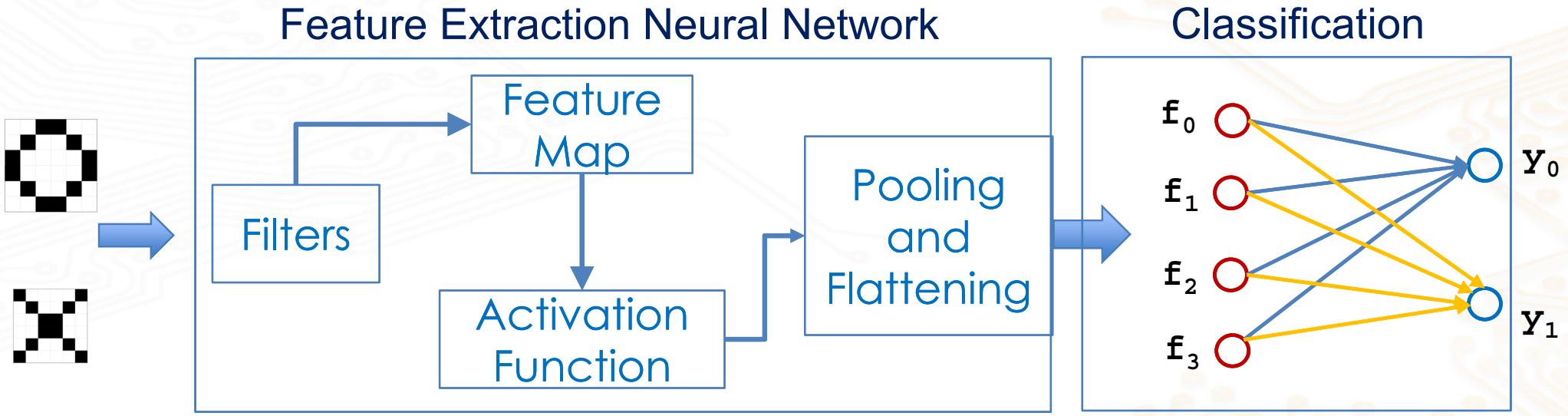
- both shapes contain (only) two basic (and common) features
  - but they are located in different places

A properly trained model would have detected these two basic features and use them to run inference.

Features Extraction and Classification

Instead of performing the classification directly from the raw image

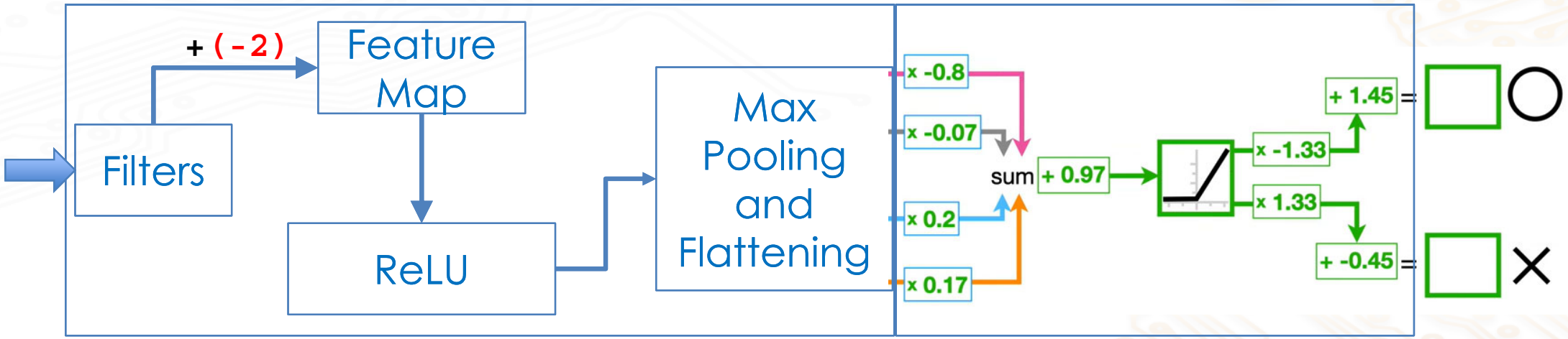
- extract distinct features from the raw image using filters (kernels)
- classified based on distinct features



A Features based Classification Model

Assuming we have a trained model that classify the ‘X’ and ‘O’ as shown below

- Filters (kernels) to be used to run inference

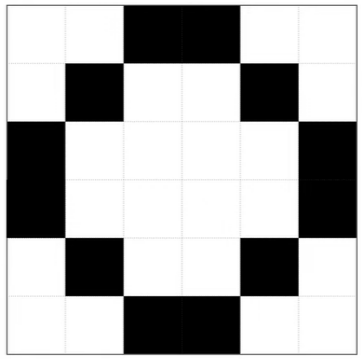


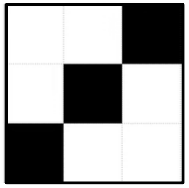
Feature Extraction Neural Network

Classification

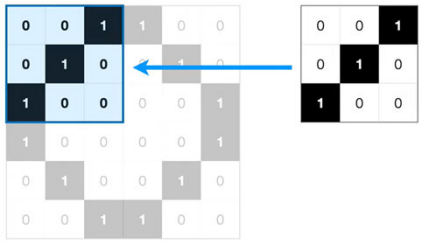
Filter/Kernel Operation (2D Conv)

Convoluted the input image with the filter/kernel





- Executing the dot-product (aka correlation)



$(0 \times 0) + (0 \times 0) + (1 \times 1)$

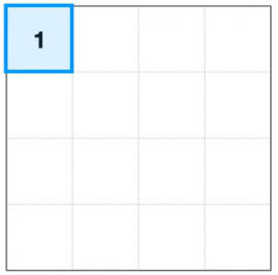
$+ (0 \times 0) + (1 \times 1) + (0 \times 0)$

$+ (1 \times 1) + (0 \times 0) + (0 \times 0)$

$= 3$

$+ (-2)$

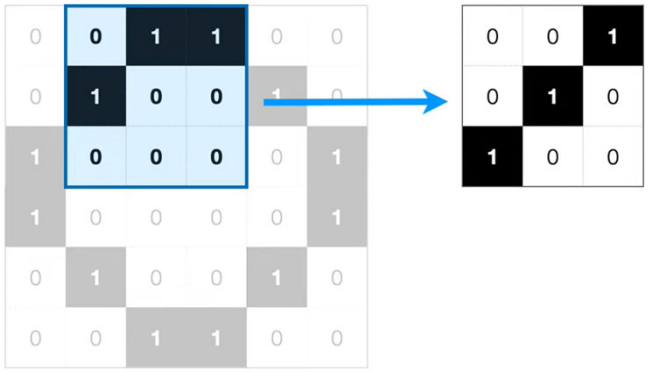
Feature Map



(Source: Youtube – StatQuest)

Filter Operation (cont.)

Shift to the next pixel

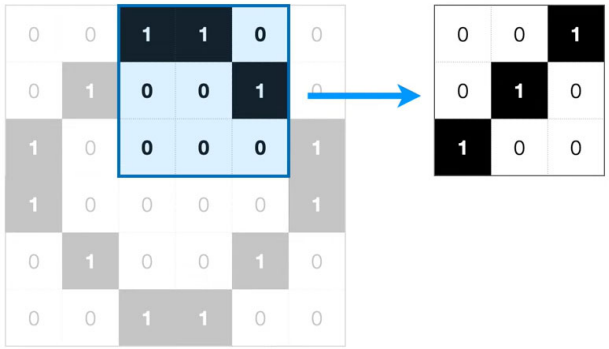


$$(0 \times 0) + (1 \times 0) + (1 \times 1)$$
$$+ (1 \times 0) + (0 \times 1) + (0 \times 0)$$
$$+ (0 \times 1) + (0 \times 0) + (0 \times 0)$$
$$= 1$$

+ (-2)

Feature Map

1	-1		



$$(1 \times 0) + (1 \times 0) + (0 \times 1)$$
$$+ (0 \times 0) + (0 \times 1) + (1 \times 0)$$
$$+ (0 \times 1) + (0 \times 0) + (0 \times 0)$$
$$= 0$$

+ (-2)

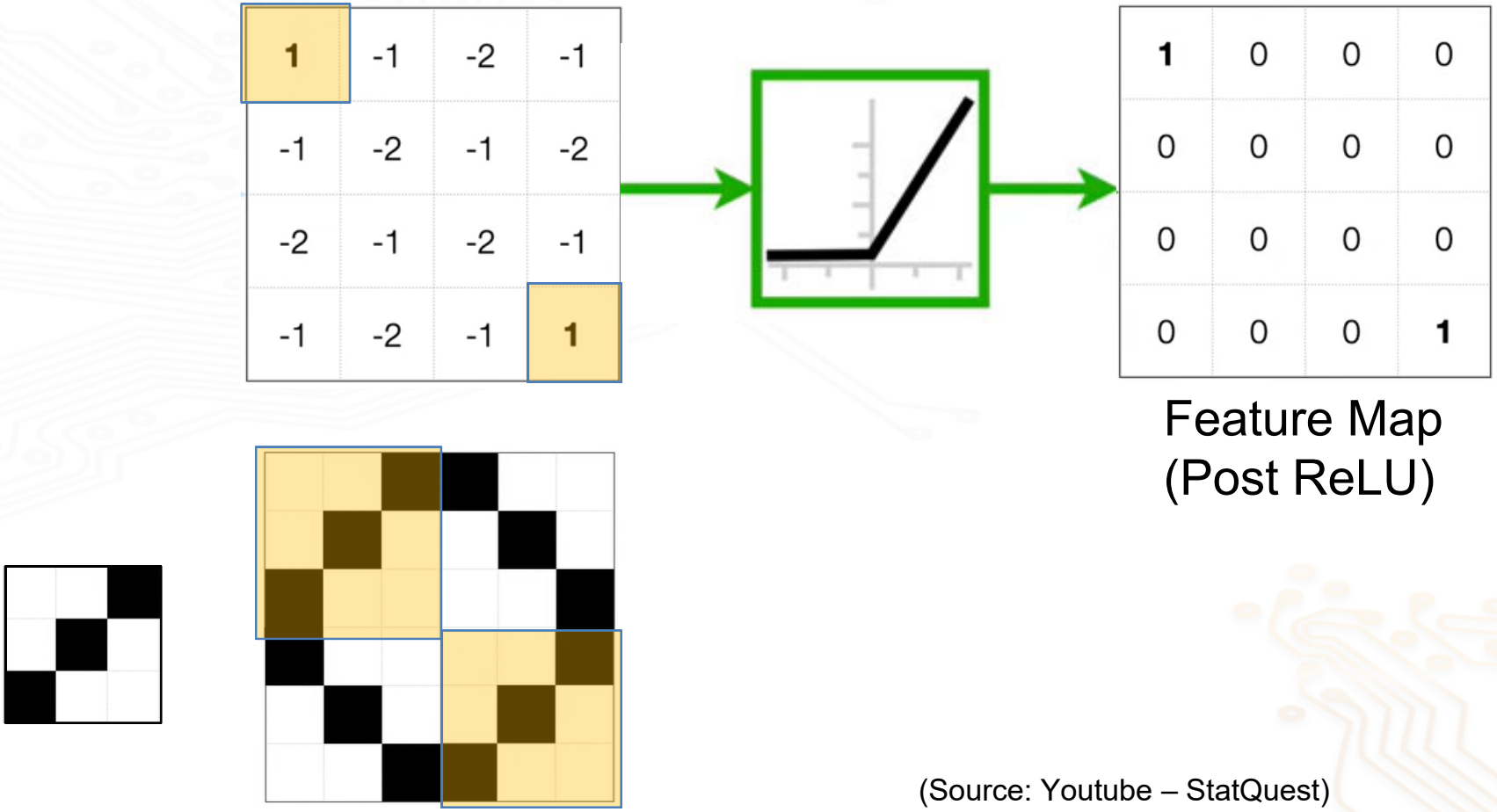
Feature Map

1	-1	-2	

(Source: Youtube – StatQuest)

Feature Map

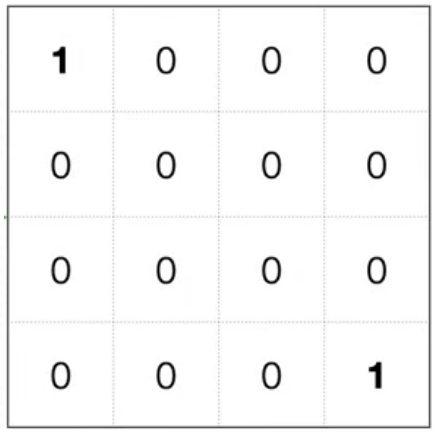
Applied through a ReLU



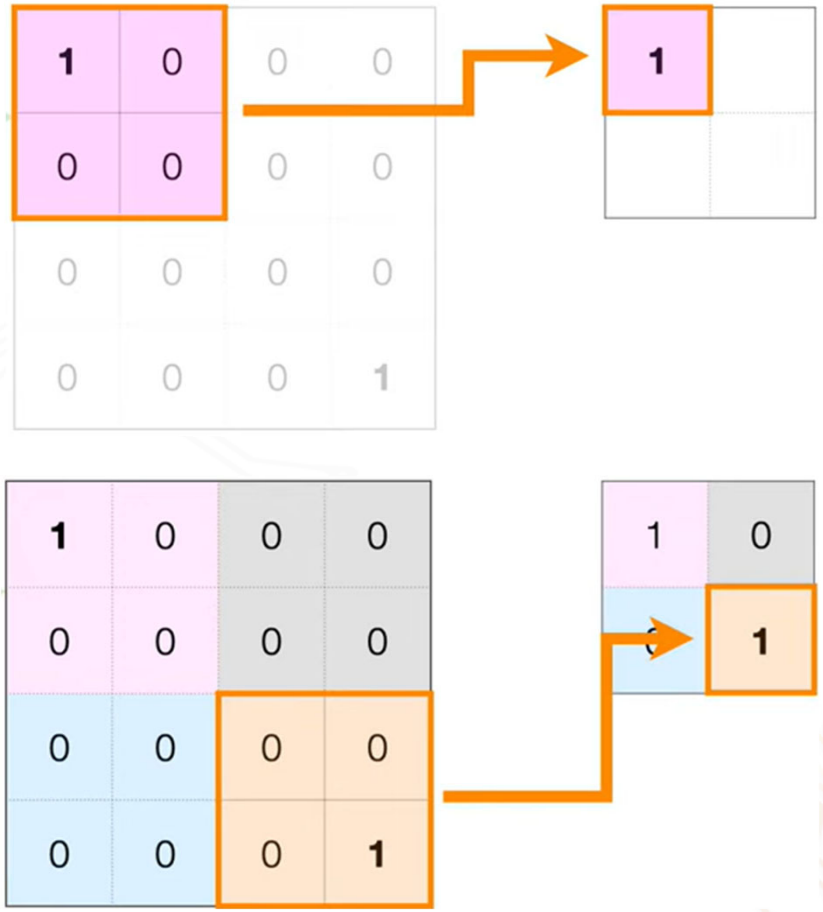
(Source: Youtube – StatQuest)

Max Pooling Feature Map

Down sampling the features map using Max pooling

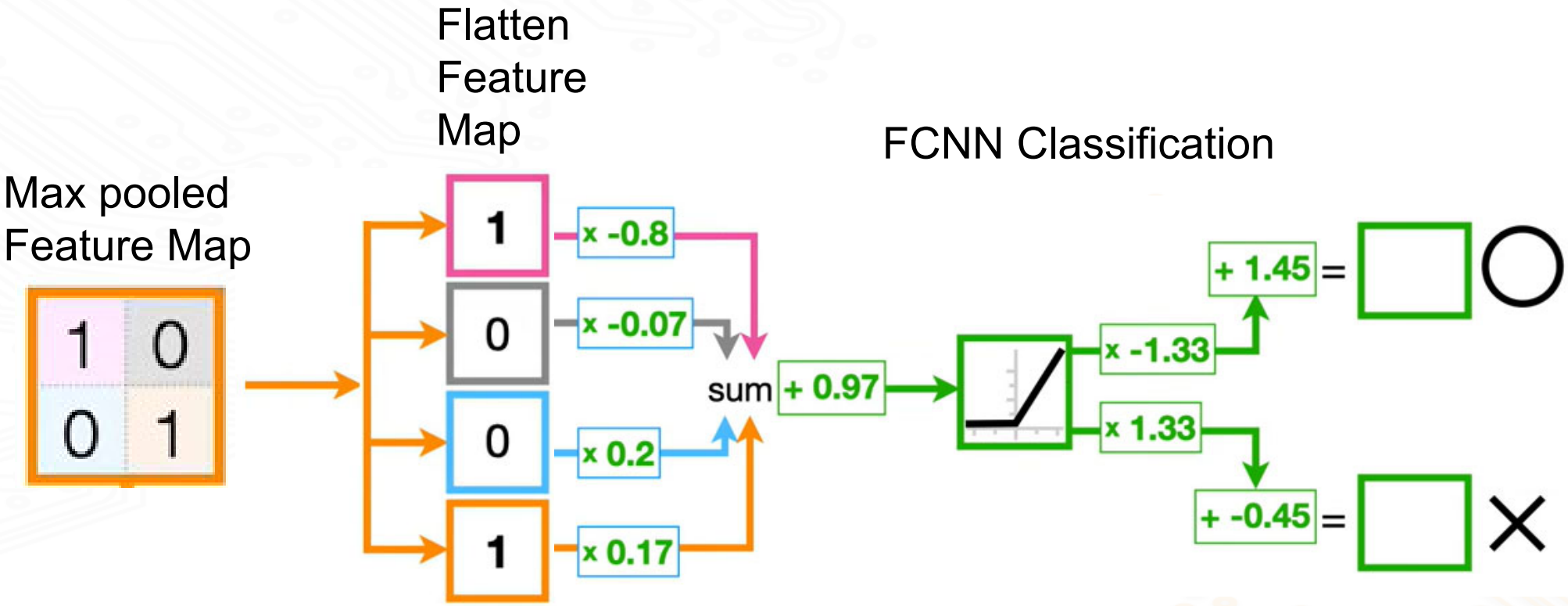


Feature Map
(Post ReLU)



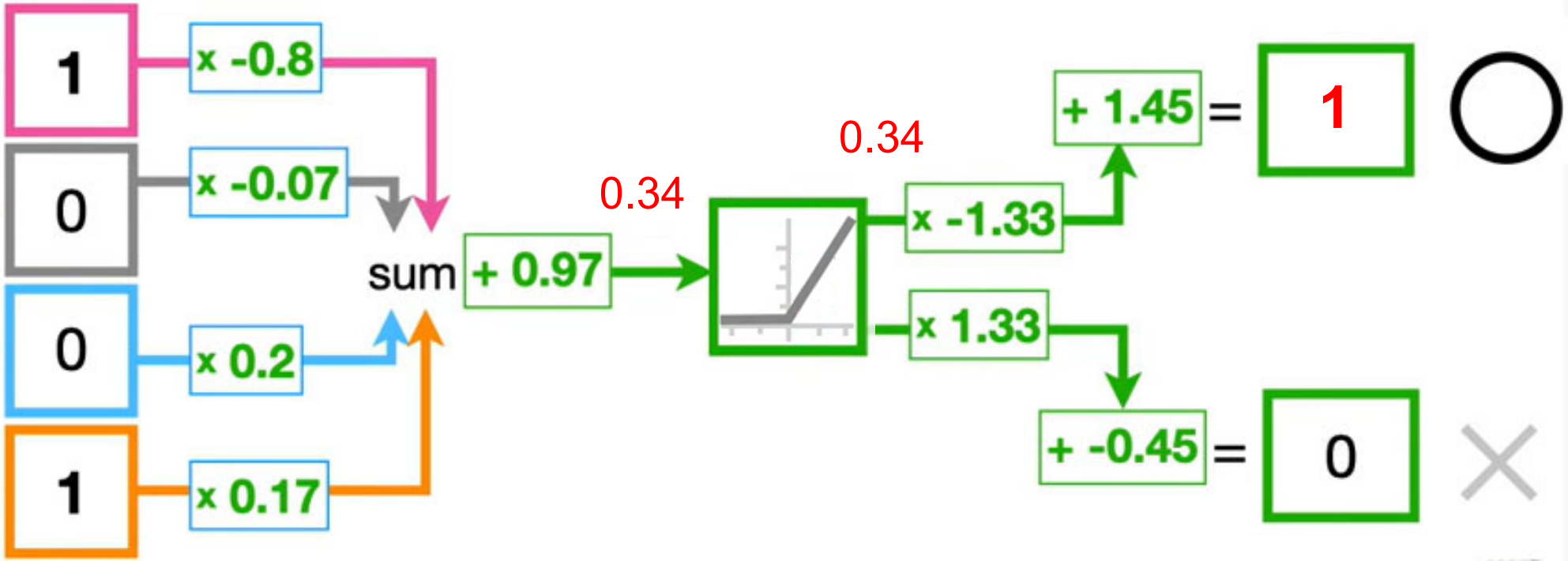
(Source: Youtube – StatQuest)

Flatten for ANN Classification



(Source: Youtube – StatQuest)

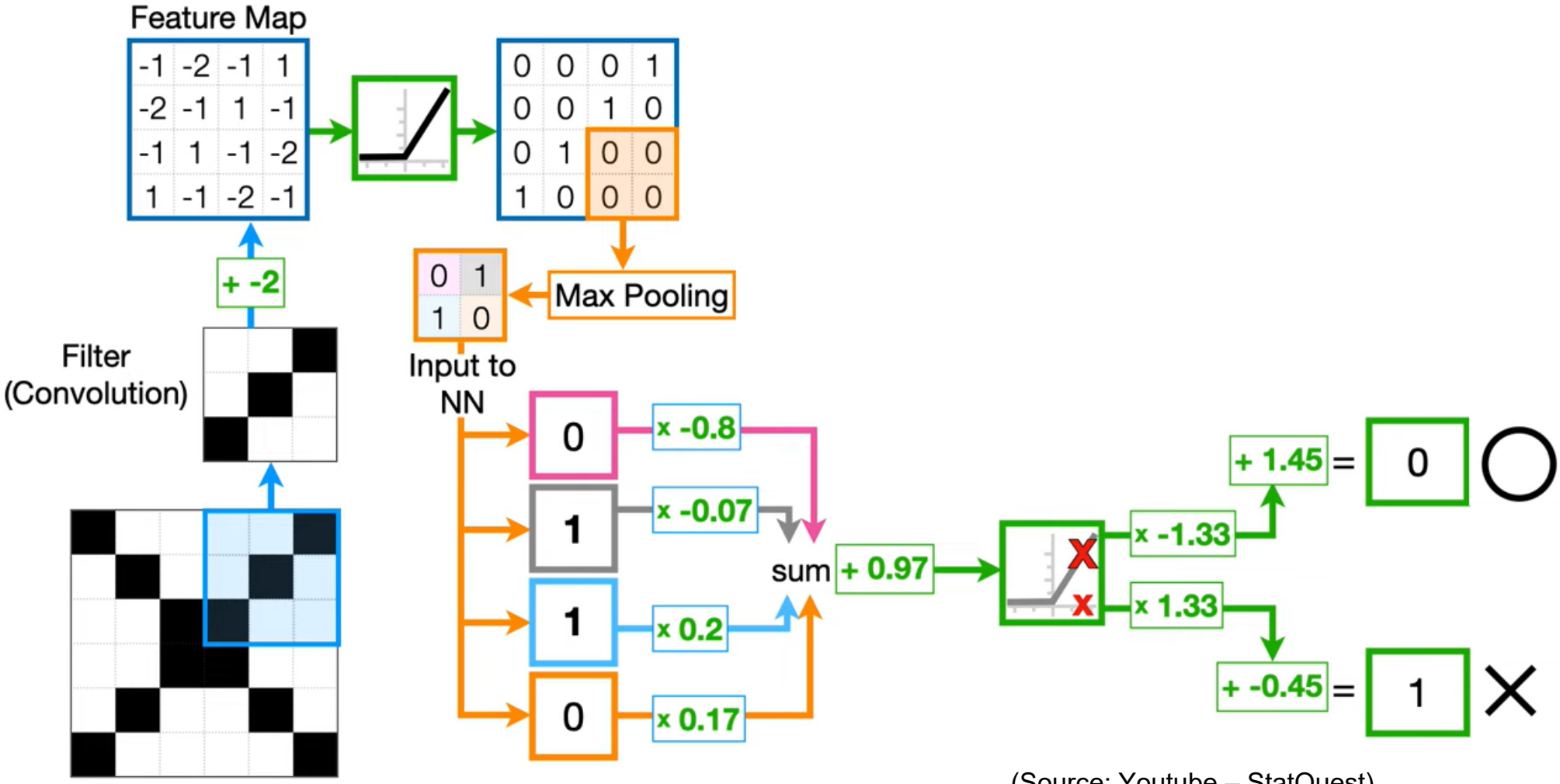
Classification for ‘O’



$$(1 \times -0.8) + (0 \times -0.07) + (0 \times 0.2) + (1 \times 0.17) + 0.97 = 0.34$$

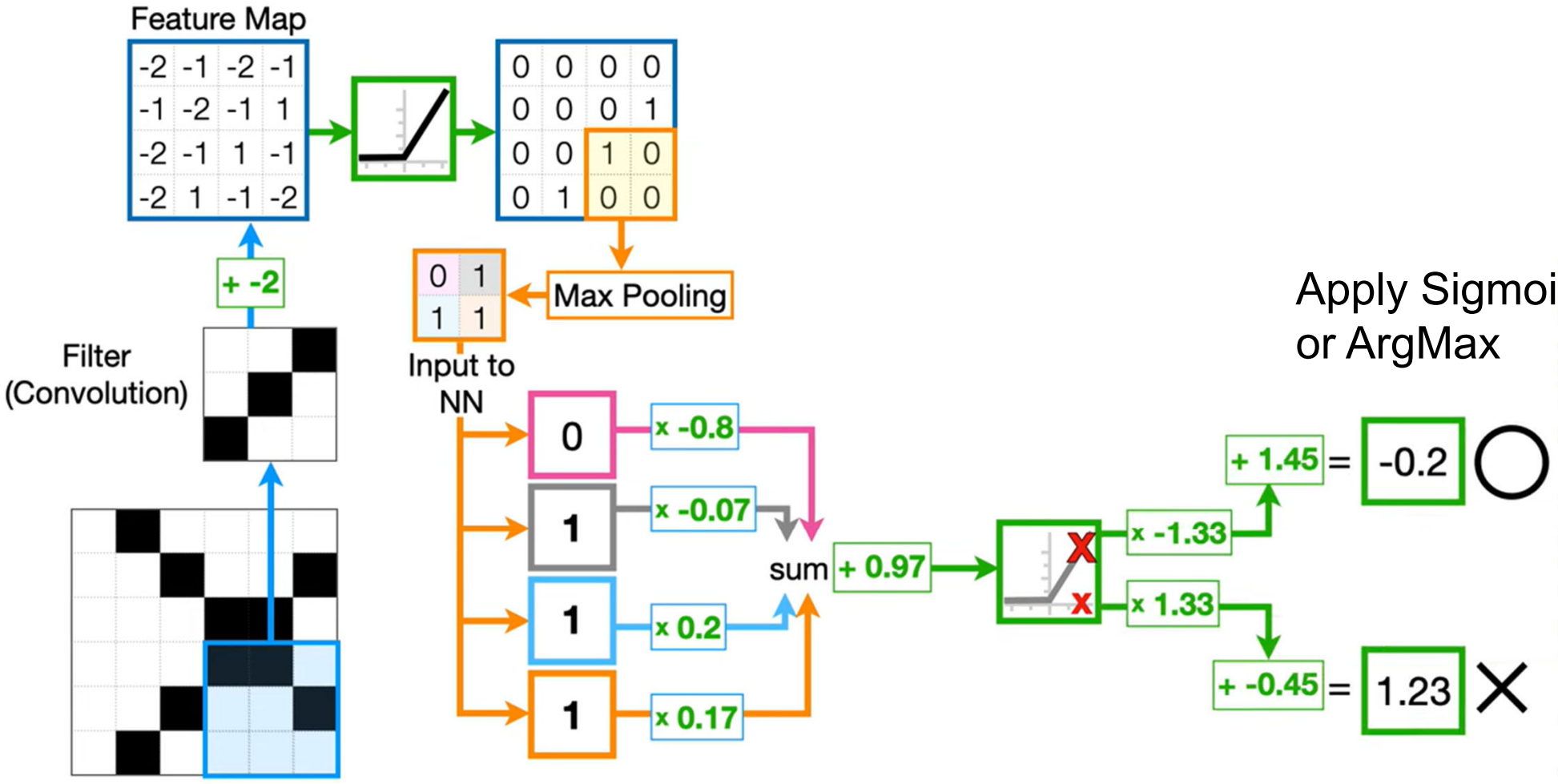
(Source: Youtube – StatQuest)

Classification for ‘X’



(Source: Youtube – StatQuest)

Classification for shifted ‘X’

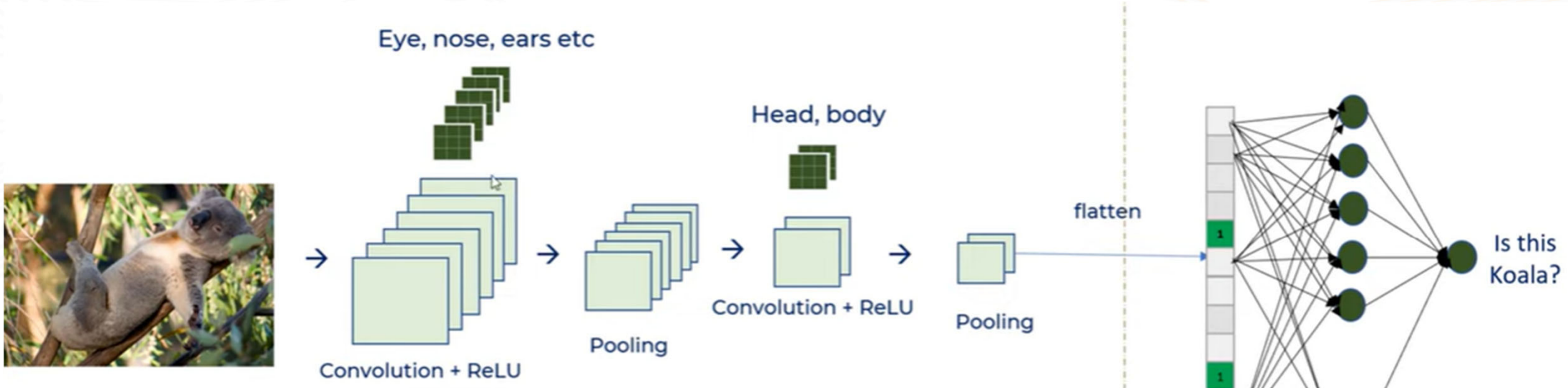


(Source: Youtube – StatQuest)

Practical Structure

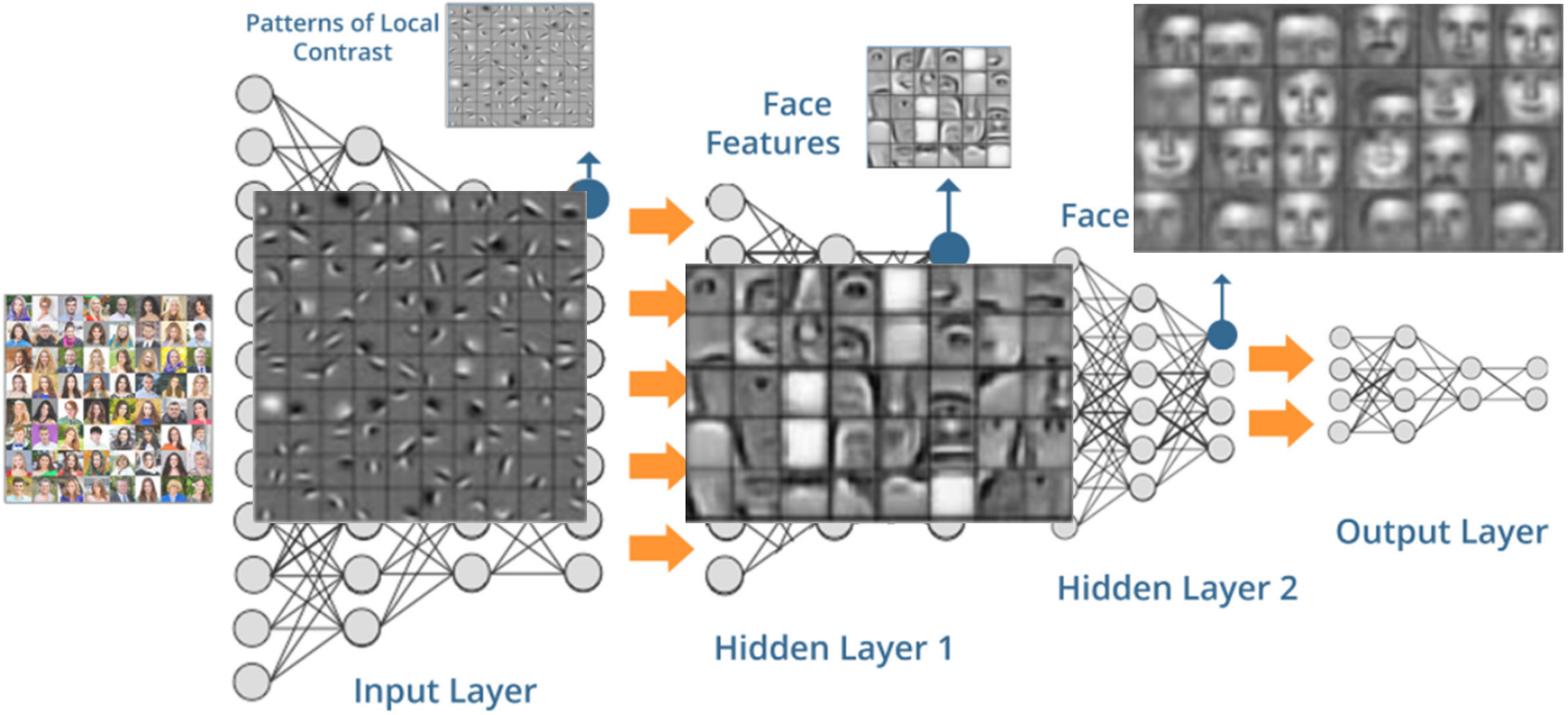
In practice

- multiple levels of convolution and pooling layers are used
  - Convolutional Neural Network - CNN
- each level extract basic features, which are analysed by the subsequent layer(s)



(Source: Youtube – codebasics)

Face Detection Training



CNN for MNIST Image Recognition

We will use Keras and TF to define a CNN model to recognize MNIST digit

- Import all necessary library

```
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt
import numpy as np
```

- Load the built-in MNIST dataset through Keras

```
(X_train, y_train) , (X_test, y_test) = keras.datasets.mnist.load_data()
```

```
X_train.shape[0]
```

```
]: 60000
```

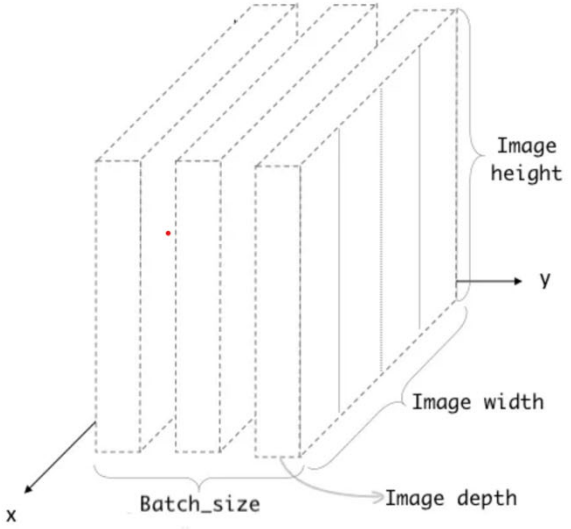
```
X_test.shape[0]
```

```
]: 10000
```

CNN for MNIST Image Recognition

Reshaping the data to 4-dimension needed by Keras API

```
#Keras API, we need 4-dims NumPy arrays
# Reshaping the array to 4-dims so that it can work with the Keras API
X_train = X_train.reshape(X_train.shape[0], 28, 28, 1)
X_test = X_test.reshape(X_test.shape[0], 28, 28, 1)
input_shape = (28, 28, 1)
```



CNN input: 4D array with a shape of (batch_size, height, width, depth)

Height and width = image size

depth = channel: color = 3, greyscale = 1

CNN for MNIST Image Recognition

Reshaping the data to 4-dimension needed by Keras KPI

```
print('X_train shape:', X_train.shape)
print('Number of images in X_train', X_train.shape[0])
print('Number of images in X_test', X_test.shape[0])
```

```
X_train shape: (60000, 28, 28, 1)
Number of images in X_train 60000
Number of images in X_test 10000
```

```
# Normalizing the RGB codes
X_train /= 255
X_test /= 255
```

CNN for MNIST Image Recognition

Import the required Keras modules

```
# Importing the required Keras modules containing model and layers
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense, Conv2D, Dropout, Flatten, MaxPooling2D
```

Define the CNN model (using model.add)

```
# Creating a Sequential Model and adding the layers
model = Sequential()
model.add(Input(shape=input_shape))
model.add(Conv2D(28, kernel_size=(3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Flatten()) # Flattening the 2D arrays for fully connected layers
model.add(Dense(128, activation=tf.nn.relu)) #add another hidden layer
model.add(Dropout(0.2)) #dropout layers to reduce overfitting
model.add(Dense(10,activation=tf.nn.softmax))
```

CNN for MNIST Image Recognition

Check the CNN model: model.summary

```
model.summary()
```

```
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                 Output Shape              Param #
=================================================================
 conv2d_1 (Conv2D)            (None, 26, 26, 28)        280

 max_pooling2d_1 (MaxPooling  (None, 13, 13, 28)       0
 2D)

 flatten_1 (Flatten)          (None, 4732)              0

 dense_2 (Dense)              (None, 128)               605824

 dropout_1 (Dropout)          (None, 128)               0

 dense_3 (Dense)              (None, 10)                1290

=================================================================
Total params: 607,394
Trainable params: 607,394
Non-trainable params: 0
_________________________________________________________________
```

CNN for MNIST Image Recognition

Specify the optimizer algorithm and measurement parameters

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

Train the model (with 10% of the data reserved for validation)

```
model.fit(x=X_train,y=y_train, epochs=5, validation_split=0.1)
#Model is being trained on 1875 batches of 32 images each (default batch size)
#1875*32 = 60000 images
#Validation = 10%
#Training = 1875 * 0.9 = 1688
```

CNN for MNIST Image Recognition

Run model against Test data to evaluate accuracy of model

```
#evaulate performance based on test data
baseline_model_accuracy = model.evaluate(X_test, y_test)
print('Baseline test accuracy:', baseline_model_accuracy)
```

Get the predicted output values

```
y_predicted = model.predict(X_test)
```

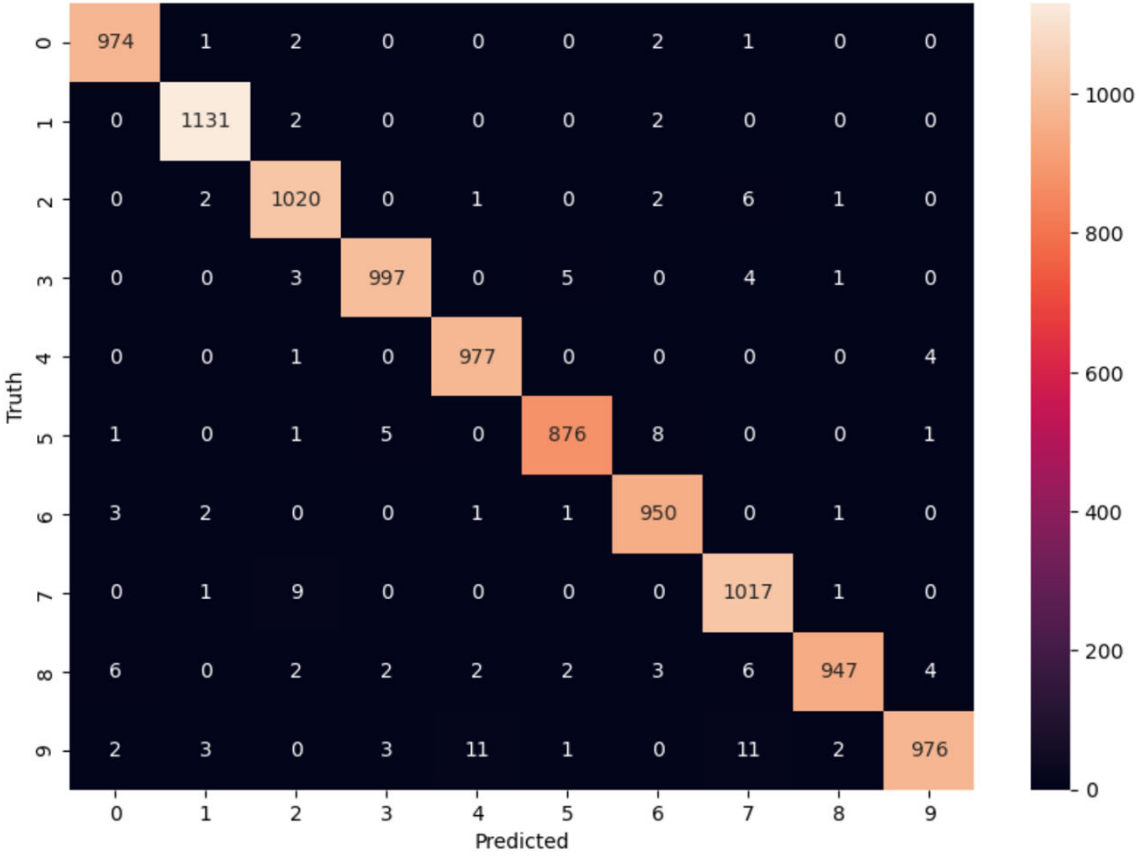
CNN for MNIST Image Recognition

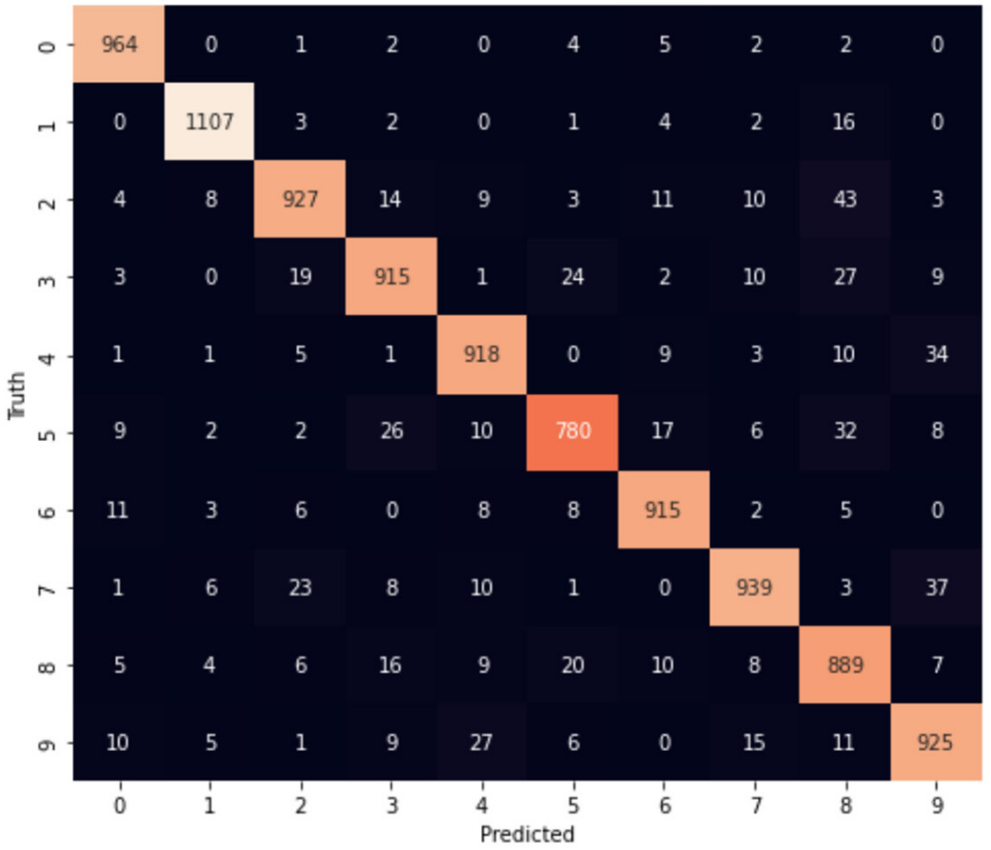
Generate the confusion matrix and heatmap

```
y_predicted_labels = [np.argmax(i) for i in y_predicted]
cm = tf.math.confusion_matrix(labels=y_test,predictions=y_predicted_labels)
import seaborn as sns
plt.figure(figsize = (10,7))
sns.heatmap(cm,annot=True, fmt='d')
plt.xlabel('Predicted')
plt.ylabel('Truth')
```

CNN for MNIST Image Recognition

Compare against the FCNN heatmap





Summary

Model architecture design is importance to the performance of ML

- particularly so for TinyML applications due to resource constraints of devices
- but no formula on how to choose the optimum design

Best starting point

- use some existing architectures that have been proven
- iteratively increase the complexity until the performance meets the specifications or requirements.

Computer Vision ⇒ Convolutional Neural Networks (CNN)

- have characteristics that enable invariance to the affine transformations of images that are fed through the network.
- provides the ability to recognize patterns that are shifted, tilted or slightly warped within images